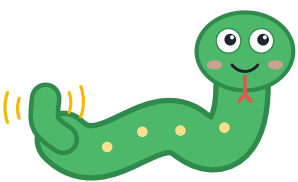
Lecture 1

Part 1: Intro and Python Basics

Hello, Python!

What programming is, why Python, AI and LLMs, the building blocks of code, and your first program



Dror Jacoby
Tel Aviv University

What is this course?

- A first course in programming, for **everyone**
- No experience needed. Really, zero.
- You learn to **think like a programmer**: break a problem into clear steps
- You write real Python: from `print("Hello")` to charts and machine learning
- You learn to work **with** AI tools, and to check what they write



Who am I?

Dror Jacoby

- Ph.D. in Electrical Engineering
- Tel Aviv University & CU
- I use Python every day in my work

drorjacoby@mail.tau.ac.il



And who are you?

- What is your name and what do you study?
- Why did you choose this course?
- Have you ever written code? (“Never” is a perfect answer)
- What would you love to build with code?



You are welcome to share with me, at any point

Questions, ideas, feedback, or something that is hard for you: in class, after class, or by email.

Course structure

10 lectures in 4 parts

Part 1	Intro and Python Basics	L1–L3
Part 2	Decisions and Loops	L4–L6
Part 3	Functions and Dictionaries	L7–L8
Part 4	Data Science and AI	L9–L10

Every lecture: examples, Try it!, Bug Zoo, AI Corner.
Every part ends with a recap and a showcase project.
Recitations: practice, worked problems, exam-style questions.

3 homeworks, one a month

HW1 Intro	∼Nov 20
HW2 Functions	∼Dec 20
HW3 Project + talk	last week

Grade

Final exam	**50%**
HW (10 + 10 + 20)	**40%**
Class participation	**10%**

Where we are

Part 1: Intro and Python Basics			Part 2: Decisions and Loops			Part 3: Functions and Dictionaries		Part 4: Data Science and AI	
L1 Hello, Python!	**L2** Variables & Types	**L3** Input & Output	**L4** Making Decisions	**L5** Lists & Sets	**L6** Loops	**L7** Functions	**L8** Dicts & Debugging	**L9** Objects, NumPy & Pandas	**L10** Plots & ML

You are here

You already write programs. Seriously.

- A recipe: exact steps, in order

You already write programs. Seriously.

- A recipe: exact steps, in order
- Directions to your apartment: turn left, then right

You already write programs. Seriously.

- A recipe: exact steps, in order
- Directions to your apartment: turn left, then right
- A WhatsApp rule: “if Mom calls, answer”

You already write programs. Seriously.

- A recipe: exact steps, in order
- Directions to your apartment: turn left, then right
- A WhatsApp rule: “if Mom calls, answer”
- That is all a program is: **clear steps**



Today's plan

◎ By the end of today you can...

- Explain what a program and an algorithm are
- Explain in plain words what AI and LLMs do
- Name the building blocks of code and run your first one



The secret to passing

Programming is learned by doing, not by watching.



Practice every week



Errors are normal



Ask questions early

Today's outline

1. **What is programming?**
   A literal intern. Programs, algorithms, how a computer works
2. **Why Python?**
   Simple to read, huge community, used from science to the web
3. **AI, LLMs and programming**
   What ChatGPT-style models do, where they fail, why you still need code
4. **The building blocks of code**
   Input, output, math, decisions, repetition
5. **Meet Google Colab**
   Your first program, `print`, running in the browser



Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- Computers do the boring, repetitive work for you, in seconds
- Every field now has data: economics, psychology, biology, law
- AI writes code too. You need to read it, check it and fix it

Where you will see it

- Apps and websites you use every day
- Research: from medicine to the social sciences
- Excel-style work done 100 times faster





Topic 1

What is programming?

Lecture 1 • Hello, Python!



TODAY

1 **What is programming?**

2 Inside a computer

3 Why Python?

4 AI and LLMs

5 Building blocks

6 Meet Google Colab

Meet your new intern: the computer

- Works incredibly fast
- Never gets bored or tired
- Does **exactly** what you say

Meet your new intern: the computer

- Works incredibly fast
- Never gets bored or tired
- Does **exactly** what you say
- …and nothing you did not say



The intern takes you literally

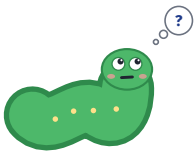
A classic joke

“Go buy a carton of milk. If they have eggs, get six.”

The intern takes you literally

A classic joke

“Go buy a carton of milk. If they have eggs, get six.”



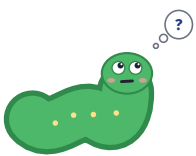
The intern comes back with **six cartons of milk**.

They had eggs, after all.

The intern takes you literally

A classic joke

"Go buy a carton of milk. If they have eggs, get six."

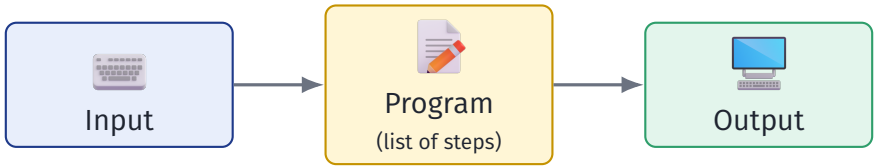


The intern comes back with **six cartons of milk**.

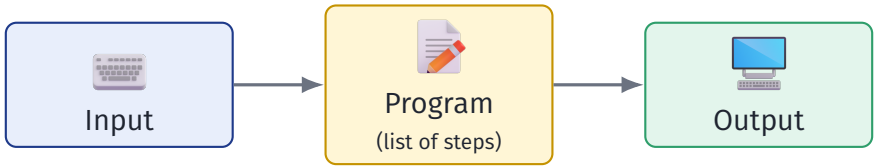
They had eggs, after all.

Computers need instructions with zero guessing.

What is a program?



What is a program?



- **Program**: a list of instructions, run in order
- **Programming**: writing those instructions in a language

Low-level or high-level: who does the hard work?

- Deep down, a laptop only understands **0s and 1s**
- **Low-level** languages (assembly, C) stay close to that – powerful, but you spell out every tiny step
- **High-level** languages (Python) read close to English – Python does the tedious translating for you

```
01001000
01101001

00100001
```

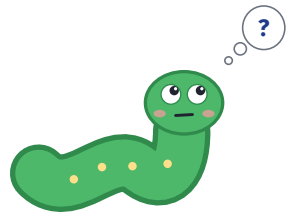
⇓ Python translates

```
print("Hi!")
```

Both ends run on the same hardware; high-level languages just hide the boring part

A short history of Python

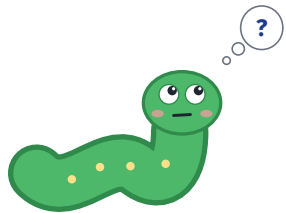
- **1989**: Guido van Rossum, bored over Christmas break at a Dutch research lab, starts a hobby project



Sources: python-course.eu, Codecademy, python.org

A short history of Python

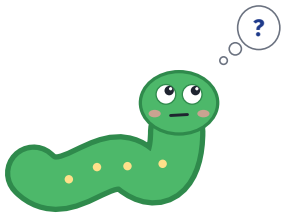
- **1989**: Guido van Rossum, bored over Christmas break at a Dutch research lab, starts a hobby project
- **1991**: first public release, version 0.9.0



Sources: python-course.eu, Codecademy, python.org

A short history of Python

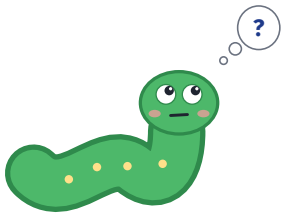
- **1989**: Guido van Rossum, bored over Christmas break at a Dutch research lab, starts a hobby project
- **1991**: first public release, version 0.9.0
- Named after *Monty Python's Flying Circus* – a comedy show, not the snake (Pip is just for fun)



Sources: python-course.eu, Codecademy, python.org

A short history of Python

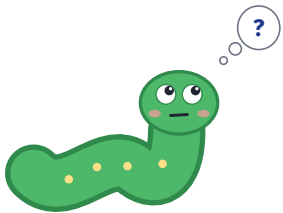
- **1989**: Guido van Rossum, bored over Christmas break at a Dutch research lab, starts a hobby project
- **1991**: first public release, version 0.9.0
- Named after *Monty Python's Flying Circus* – a comedy show, not the snake (Pip is just for fun)
- **1994, 2000, 2008**: versions 1.0, 2.0, 3.0



Sources: python-course.eu, Codecademy, python.org

A short history of Python

- **1989**: Guido van Rossum, bored over Christmas break at a Dutch research lab, starts a hobby project
- **1991**: first public release, version 0.9.0
- Named after *Monty Python's Flying Circus* – a comedy show, not the snake (Pip is just for fun)
- **1994, 2000, 2008**: versions 1.0, 2.0, 3.0
- Friends once joked Guido was the project's "Benevolent Dictator for Life" – the nickname stuck for years



Sources: python-course.eu, Codecademy, python.org

Python connects everything

- Still No. 1 on the TIOBE index (Sept. 2026, 17.8%) – far ahead of #2
- Websites, data, automation, robots, AI – all the same language
- Often the “glue”: Python code calls fast C/C++ libraries underneath, and talks to databases, APIs and the cloud



Source: TIOBE Index, September 2026 (tiobe.com/tiobe-index)

Programs are like recipes

Recipe	Program
Ingredients	Input (data)
Steps	Instructions
“Bake 20 min”	One command
The cake	Output



Skip a step and the cake fails.
Same with code.

How fast is the intern?

- A laptop does **billions** of steps per second
- Count to a billion out loud: about 30 years

How fast is the intern?

- A laptop does **billions** of steps per second
- Count to a billion out loud: about 30 years
- Fast, but not smart
- The smart part is **you**



30 years: one number per second, nonstop

What is an algorithm?

Algorithm

A recipe for solving a problem that **always** works.

Example: find the tallest student in a line

What is an algorithm?

Algorithm

A recipe for solving a problem that **always** works.

Example: find the tallest student in a line

1. Remember the first student as “tallest so far”

What is an algorithm?

Algorithm

A recipe for solving a problem that **always** works.

Example: find the tallest student in a line

1. Remember the first student as “tallest so far”
2. Walk to the next student

What is an algorithm?

Algorithm

A recipe for solving a problem that **always** works.

Example: find the tallest student in a line

1. Remember the first student as “tallest so far”
2. Walk to the next student
3. Taller than “tallest so far”? Remember this one instead

What is an algorithm?

Algorithm

A recipe for solving a problem that **always** works.

Example: find the tallest student in a line

1. Remember the first student as “tallest so far”
2. Walk to the next student
3. Taller than “tallest so far”? Remember this one instead
4. Repeat until the line ends. Done!

Your turn: be the programmer

Put the steps in order

A Pour hot water into the cup
B Boil water
C Put a tea bag in the cup
D Fill the kettle



Your turn: be the programmer

Put the steps in order

A Pour hot water into the cup
B Boil water
C Put a tea bag in the cup
D Fill the kettle



Answer

D, B, C, A (C, then B also works!)

Topic 2

Inside a computer

Lecture 1 • Hello, Python!



TODAY

1. What is programming?
2. **Inside a computer**
3. Why Python?
4. AI and LLMs
5. Building blocks
6. Meet Google Colab

Every computer does three things



Input	Processing	Output

Every computer does three things



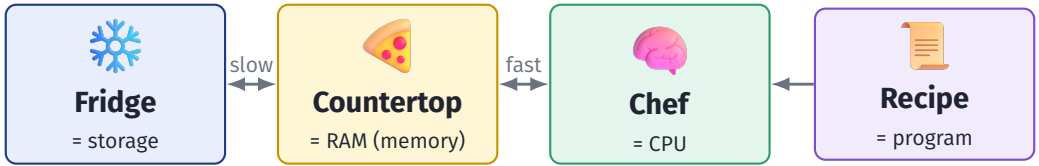
Input	Processing	Output
You type and tap Send	The phone packs and sends it	Two blue ticks ✓✓

Every computer does three things

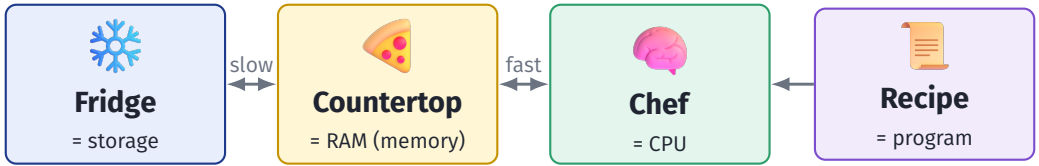


Input	Processing	Output
You type and tap Send	The phone packs and sends it	Two blue ticks ✓✓
You tap your Rav-Kav	The reader checks and takes the fare	Beep and a green light

Inside the box: a kitchen

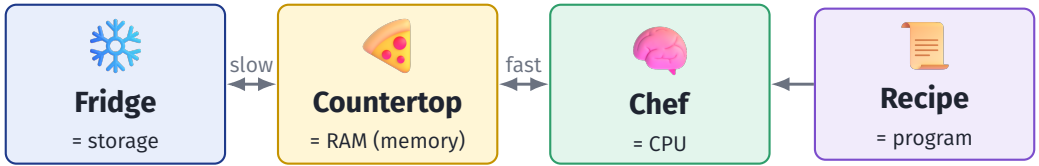


Inside the box: a kitchen



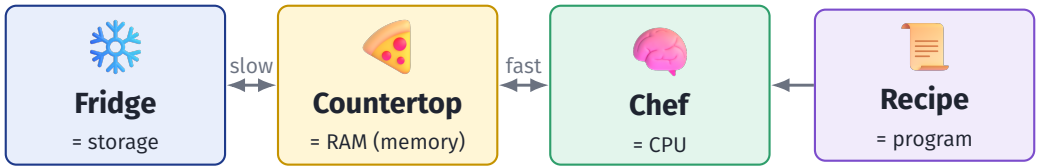
- **Fridge (storage)**: big and slow, keeps your files even when the power is off

Inside the box: a kitchen



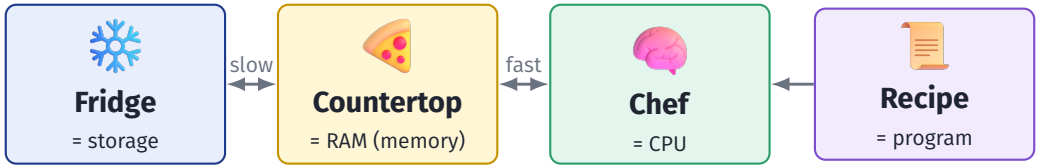
- **Fridge (storage)**: big and slow, keeps your files even when the power is off
- **Countertop (RAM)**: small and fast, holds what you use *right now*, cleared at shutdown

Inside the box: a kitchen



- **Fridge (storage)**: big and slow, keeps your files even when the power is off
- **Countertop (RAM)**: small and fast, holds what you use *right now*, cleared at shutdown
- **Chef (CPU)**: does the actual work, one tiny step after another

Inside the box: a kitchen



- **Fridge (storage)**: big and slow, keeps your files even when the power is off
- **Countertop (RAM)**: small and fast, holds what you use *right now*, cleared at shutdown
- **Chef (CPU)**: does the actual work, one tiny step after another
- **Recipe (program)**: tells the chef what to do. That is the part **you** write!

From your Python to the chef



From your Python to the chef



- Each CPU step is very simple, but it does **billions per second**

From your Python to the chef



- Each CPU step is very simple, but it does **billions per second**
- Remember the 0s and 1s? The translating is Python's job, not yours

From your Python to the chef



- Each CPU step is very simple, but it does **billions per second**
- Remember the 0s and 1s? The translating is Python's job, not yours
- You think in Python. The chef only needs tiny, exact steps

Why this matters for us

- The chef follows the recipe **exactly**, like our literal intern



Why this matters for us

- The chef follows the recipe **exactly**, like our literal intern
- It never guesses what you meant



Why this matters for us

- The chef follows the recipe **exactly**, like our literal intern
- It never guesses what you meant
- A wrong dish? The problem is in the **recipe**, so we fix the code



Why this matters for us

- The chef follows the recipe **exactly**, like our literal intern
- It never guesses what you meant
- A wrong dish? The problem is in the **recipe**, so we fix the code
- That is great news: **you** are in control



Why this matters for us

- The chef follows the recipe **exactly**, like our literal intern
- It never guesses what you meant
- A wrong dish? The problem is in the **recipe**, so we fix the code
- That is great news: **you** are in control



The computer does exactly what the code says.

Which part is it?

❓ Input, output, CPU, RAM or storage?

1. The screen shows your friend’s message
2. Your photos, still there after a restart
3. The browser tabs you are using right now
4. Billions of tiny steps every second



Which part is it?

Input, output, CPU, RAM or storage?

1. The screen shows your friend's message
2. Your photos, still there after a restart
3. The browser tabs you are using right now
4. Billions of tiny steps every second



Answer

1. Output
2. Storage (fridge)
3. RAM (countertop)
4. CPU (chef)

Topic 3

Why Python?

Lecture 1 • Hello, Python!



TODAY

1. What is programming?
2. Inside a computer
3. **Why Python?**
4. AI and LLMs
5. Building blocks
6. Meet Google Colab

Meet Python

- Created by Guido van Rossum, first released 1991
- Named after the comedy show *Monty Python*

Meet Python

- Created by Guido van Rossum, first released 1991
- Named after the comedy show *Monty Python*
- Not the snake! (We adopted one anyway: Pip)
- Free and open source



Python reads almost like English

Java

```java
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hi!");
  }
}
```

Python

```python
print("Hi!")
```

Python reads almost like English

Java

```java
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hi!");
  }
}
```

Python

```python
print("Hi!")
```

One line. That's it.

Less typing, fewer places to make mistakes.

Python is everywhere

- No. 1 on the TIOBE language ranking (Sept. 2026)
- Used by beginners **and** by giant companies
- Huge free library of ready-made tools
- Friendly community: answers for every question



Source: TIOBE Index, September 2026 (tiobe.com/tiobe-index)

Where is Python used?



Websites
Instagram's backend



AI
chatbots, translation



Data
charts and reports



Space
NASA mission tools



Music
song recommendations



Science
biology, physics



Games
game scripts



Finance
prices and risk

Python in *your* field

- 📚 **Humanities**: count words in 1000 novels
- 💬 **Social sciences**: analyze survey answers
- 📈 **Management**: sales dashboards in minutes
- 🎨 **Arts**: generative art and music

Any job with data or repetition can use Python.

Topic 4

AI and LLMs

Lecture 1 • Hello, Python!



TODAY

1 What is programming?

2 Inside a computer

3 Why Python?

4 AI and LLMs

5 Building blocks

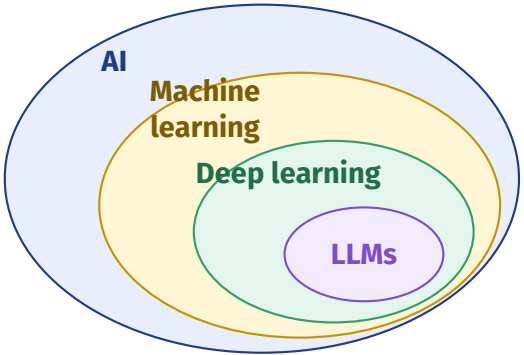
6 Meet Google Colab

AI is already in your pocket

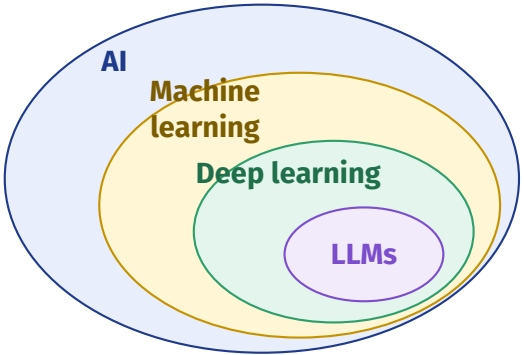
- Your phone finds “cat” photos in your gallery
- Google Translate, voice assistants
- Netflix and Spotify suggestions
- Chatbots that write text and code



AI, machine learning, LLMs: circles inside circles

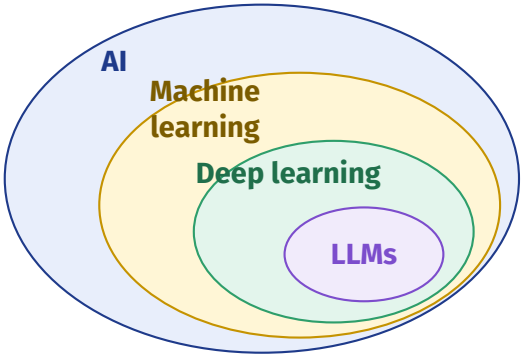


AI, machine learning, LLMs: circles inside circles



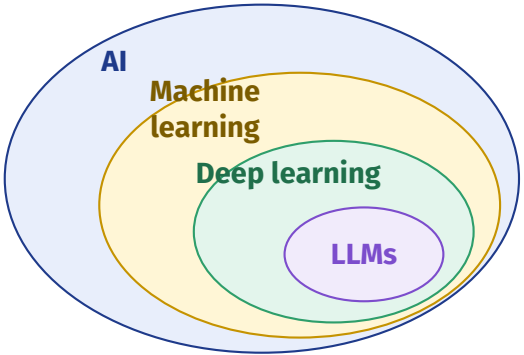
- **AI**: computers doing “smart” tasks

AI, machine learning, LLMs: circles inside circles



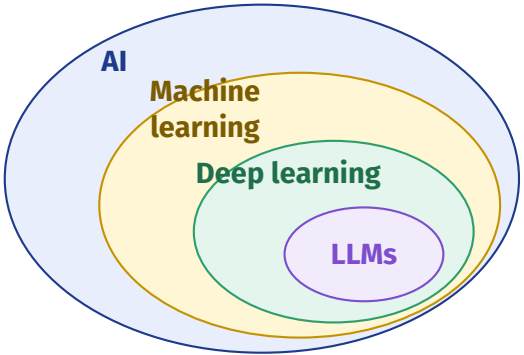
- **AI**: computers doing “smart” tasks
- **Machine learning**: learning from examples

AI, machine learning, LLMs: circles inside circles



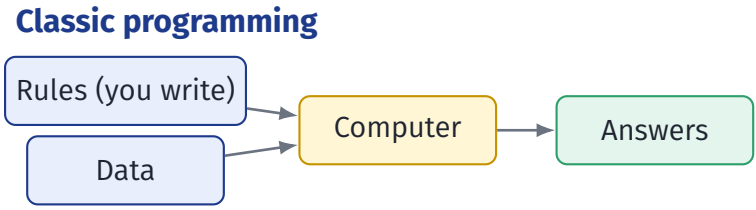
- **AI**: computers doing “smart” tasks
- **Machine learning**: learning from examples
- **Deep learning**: huge “brain-like” networks

AI, machine learning, LLMs: circles inside circles

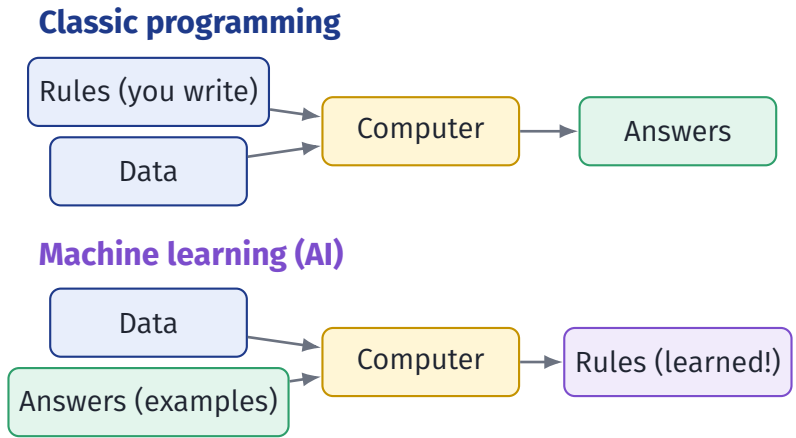


- **AI**: computers doing “smart” tasks
- **Machine learning**: learning from examples
- **Deep learning**: huge “brain-like” networks
- **LLMs**: deep learning trained on text

Two ways to make a computer smart



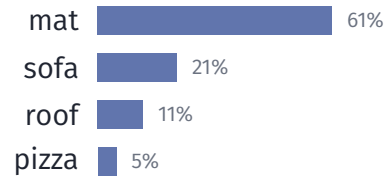
Two ways to make a computer smart



How does a chatbot write?

It guesses the **next word**, again and again.

“The cat sat on the …”



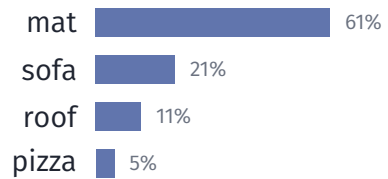
Made-up numbers, just to show the idea

How does a chatbot write?

It guesses the **next word**, again and again.

“The cat sat on the …”

It learned these guesses from huge amounts of text.



Made-up numbers, just to show the idea

What is an LLM?



Large

learned from a huge pile of text



Language

reads and writes words (and code)



Model

a giant math formula that predicts

Examples: ChatGPT, Claude, Gemini, Copilot. 🤖

LLMs read in small pieces: tokens

“Tel Aviv is sunny today”

Example split, just to show the idea. Real splits differ by model.

LLMs read in small pieces: tokens

“Tel Aviv is sunny today”



Example split, just to show the idea. Real splits differ by model.

LLMs read in small pieces: tokens

“Tel Aviv is sunny today”



- A **token** is a word or a piece of a word
- The model turns each token into numbers

Example split, just to show the idea. Real splits differ by model.

LLMs read in small pieces: tokens

“Tel Aviv is sunny today”



- A **token** is a word or a piece of a word
- The model turns each token into numbers
- Then it predicts the **next** token, one at a time

Example split, just to show the idea. Real splits differ by model.

Joke break: count the r's

Me: How many r's are in "strawberry"?

Joke break: count the r's

Me: How many r's are in "strawberry"?

Older chatbots: "There are 2 r's in strawberry."

(Confidently. Many times.)

Joke break: count the r's

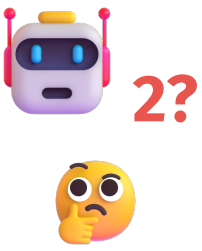
Me: How many r's are in "strawberry"?

Older chatbots: "There are 2 r's in strawberry."

(Confidently. Many times.)

Why? The model sees **tokens**, not letters: `straw` | `berry`. Counting letters is hard for it.

Python just counts: `"strawberry".count("r")` gives `3`.



How is an LLM made?



It learned **patterns** in text. It does not look things up like Google.

What LLMs are good and bad at

Good at

- Explaining ideas in simple words
- Writing a first draft of code
- Finding the bug in a short program

Weak at

- Exact facts, dates and numbers
- Knowing what is new today
- Saying “I don’t know”

Hallucination: confidently wrong

You ask

“Which course did Pip the snake teach at TAU in 1998?”

Hallucination: confidently wrong

You ask

“Which course did Pip the snake teach at TAU in 1998?”

An LLM might answer

“Pip taught *Advanced Reptile Python* in 1998.”

Hallucination: confidently wrong

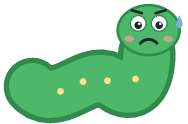
You ask

“Which course did Pip the snake teach at TAU in 1998?”

An LLM might answer

“Pip taught *Advanced Reptile Python* in 1998.”

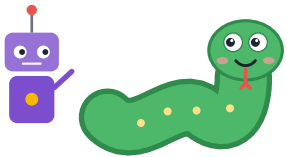
Pip didn’t exist in 1998. The answer only **sounds** right.



Made-up example of a typical mistake

AI is built with Python

- Most AI research code is written in Python
- Popular AI tools: PyTorch, TensorFlow, scikit-learn
- You will try scikit-learn in Lecture 10!



LLMs can write code

💬 You ask

“Write Python that converts 30 degrees Celsius to Fahrenheit.”

LLMs can write code

You ask

“Write Python that converts 30 degrees Celsius to Fahrenheit.”

The LLM writes

```
celsius = 30
fahrenheit = celsius * 9 / 5 + 32
print(celsius, "C is", fahrenheit, "F")
```

LLMs can write code

You ask

“Write Python that converts 30 degrees Celsius to Fahrenheit.”

The LLM writes

```
celsius = 30
fahrenheit = celsius * 9 / 5 + 32
print(celsius, "C is", fahrenheit, "F")
```

OUTPUT

```
30 C is 86.0 F
```

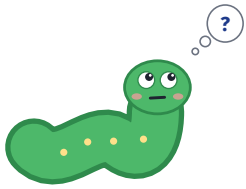
Correct! By Lecture 3 you can read every word of it.

“If AI can write code, why should I learn?”

- AI can be **confidently wrong**
- You must read and check what it writes

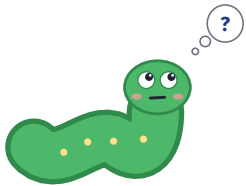
“If AI can write code, why should I learn?”

- AI can be **confidently wrong**
- You must read and check what it writes
- You need to know **what to ask**
- Understanding code = you are the boss



“If AI can write code, why should I learn?”

- AI can be **confidently wrong**
- You must read and check what it writes
- You need to know **what to ask**
- Understanding code = you are the boss



AI is a calculator for code. You still need math.

AI in this course

- AI is your **tutor**, not your ghostwriter
- Lecture 8: how to work with AI well

⚠ Course AI policy

- **OK:** ask AI to explain ideas, errors, examples
- **Not OK:** hand in code you cannot explain line by line
- Used AI in homework? Say how in a comment at the top
- The final exam is **without** AI



Topic 5

Building blocks

Lecture 1 • Hello, Python!



TODAY

1. What is programming?
2. Inside a computer
3. Why Python?
4. AI and LLMs
5. **Building blocks**
6. Meet Google Colab

Every program is built from the same bricks



Variables
L2: store a value



Input & output
L3: talk to the user



Decisions
L4: if this, do that



Lists
L5: many values



Loops
L6: repeat



Functions
L7: name a recipe



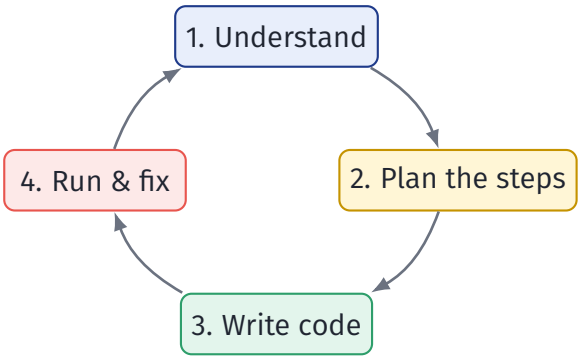
Dictionaries
L8: look things up



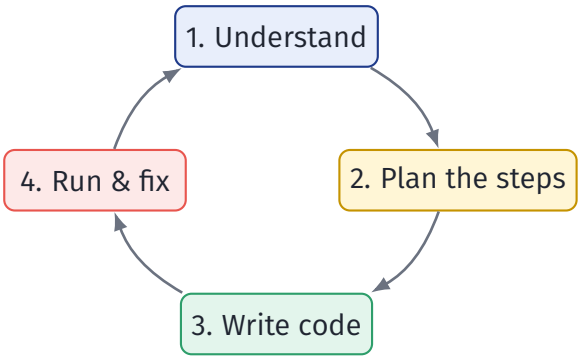
Libraries
L9-10: ready-made tools

Like Lego: a few kinds of bricks, endless things to build. 🧱

From a problem to a working program

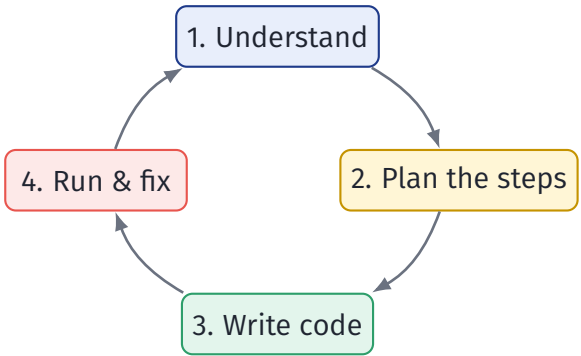


From a problem to a working program



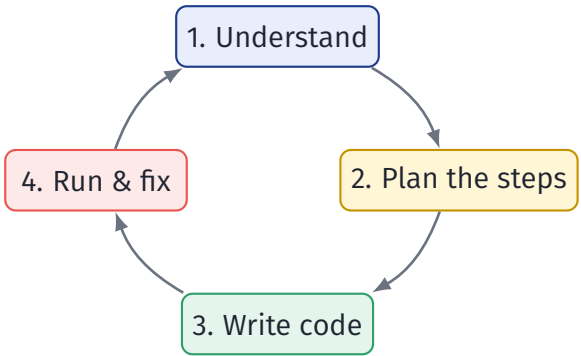
- Step 2 is the **algorithm**

From a problem to a working program



- Step 2 is the **algorithm**
- Step 4 happens a lot. That's normal!

From a problem to a working program



- Step 2 is the **algorithm**
- Step 4 happens a lot. That's normal!
- Most of the thinking is **before** typing

A tiny program with many bricks

```
prices = {"coffee": 12, "croissant": 15}
def total(order):
    result = 0
    for item in order:
        result = result + prices[item]
    return result
bill = total(["coffee", "croissant", "coffee"])
if bill > 30:
    print("Big order!", bill)
```

A tiny program with many bricks

```
prices = {"coffee": 12, "croissant": 15}
def total(order):
    result = 0
    for item in order:
        result = result + prices[item]
    return result
bill = total(["coffee", "croissant", "coffee"])
if bill > 30:
    print("Big order!", bill)
```

OUTPUT

```
Big order! 39
```

A tiny program with many bricks

```
prices = {"coffee": 12, "croissant": 15}
def total(order):
    result = 0
    for item in order:
        result = result + prices[item]
    return result
bill = total(["coffee", "croissant", "coffee"])
if bill > 30:
    print("Big order!", bill)
```

OUTPUT

```
Big order! 39
```

Line 1	dictionary
Line 2	function
Line 4	loop
Line 7	list, variable
Line 8	decision
Line 9	output

A tiny program with many bricks

```
prices = {"coffee": 12, "croissant": 15}
def total(order):
    result = 0
    for item in order:
        result = result + prices[item]
    return result
bill = total(["coffee", "croissant", "coffee"])
if bill > 30:
    print("Big order!", bill)
```

OUTPUT

```
Big order! 39
```

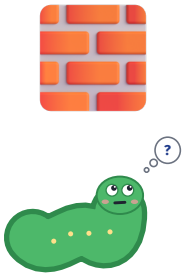
Line 1	dictionary
Line 2	function
Line 4	loop
Line 7	list, variable
Line 8	decision
Line 9	output

Looks scary? By Lecture 8 you will read it easily.

Which brick do I need?

Match the task to a brick

1. Send a reminder to all 30 students
2. Free shipping only if the order is over 200 ILS
3. Remember the user's name for later



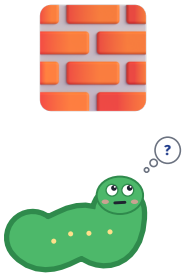
Which brick do I need?

Match the task to a brick

1. Send a reminder to all 30 students
2. Free shipping only if the order is over 200 ILS
3. Remember the user's name for later

Answer

1. Loop 2. Decision 3. Variable



Topic 6

Meet Google Colab

Lecture 1 • Hello, Python!



TODAY

1. What is programming?
2. Inside a computer
3. Why Python?
4. AI and LLMs
5. Building blocks
6. **Meet Google Colab**

Where do we write Python?

Programs on your computer

- IDLE, PyCharm, VS Code
- Install first, then write
- A demo in Lecture 10

Notebooks in the browser

- Google Colab, Jupyter
- Nothing to install

In this course we use Google Colab. 💻

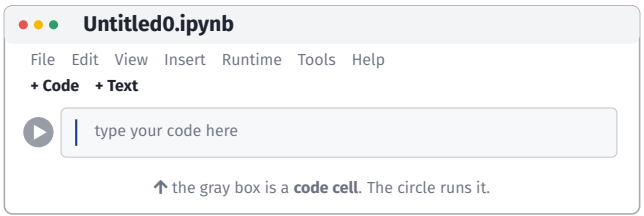
Why Colab?

- Free, runs in any browser
- Works on laptops, lab PCs, even tablets
- Saves to your Google Drive
- Has a built-in AI helper (Gemini)



Open your first notebook

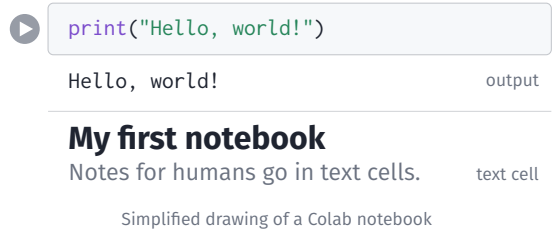
1. Go to **`colab.research.google.com`**
2. Sign in with your Google account
3. Click **New notebook**
4. Type code in the gray box, press `Shift`+`Enter`



A new Colab notebook (simplified drawing)

A notebook has two kinds of cells

- **Code cell**: Python that runs
- **Text cell**: notes for humans
- Output appears right under the cell



Simplified drawing of a Colab notebook

Your first program

L01_01_hello_world.ipynb

```
print("Hello, world!")
```

Your first program

L01_01_hello_world.ipynb

```
print("Hello, world!")
```

OUTPUT

```
Hello, world!
```

Your first program

L01_01_hello_world.ipynb

```
print("Hello, world!")
```

OUTPUT

```
Hello, world!
```

- `print` shows something on the screen
- The text goes inside `"quotes"`



print can show text and math

```
print("I love pizza")
print(2 + 3)
print("2 + 3")
```

print can show text and math

```
print("I love pizza")
print(2 + 3)
print("2 + 3")
```

OUTPUT

```
I love pizza
5
2 + 3
```

print can show text and math

```
print("I love pizza")
print(2 + 3)
print("2 + 3")
```

OUTPUT

```
I love pizza
5
2 + 3
```

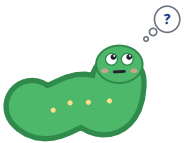
Notice

No quotes: Python calculates. With quotes: Python just copies the text.

Predict the output

Predict the output!

```
print(3 * 4)
print("3 * 4")
```



Predict the output

Predict the output!

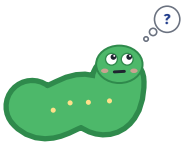
```
print(3 * 4)
print("3 * 4")
```

Answer

12

3 * 4

* means “times”.



Comments: notes for humans

```python
# This line is a comment - Python ignores it
print("Hi!")   # a comment can also go here
```

OUTPUT

```
Hi!
```

- Everything after `#` is skipped by Python
- Use comments to explain *why*, for future you

Try it!

Try it in Colab notebooks/L01/L01_01_hello_world.ipynb

1. Print your name
2. Print your favorite food
3. Print how many hours are in a week: `7 * 24`



Bug Zoo: your first bugs

BUG ZOO

```
Print("hi")
```

ERROR

```
NameError: name 'Print' is not
defined. Did you mean: 'print'?
```

Capital letters matter

Python sees `Print` and `print` as different words.

Fix: `print("hi")`

Bug Zoo: missing pieces

BUG ZOO

```
print("hi)
```

ERROR
```
SyntaxError: unterminated string literal (detected at line 1)
```

Fix: close the quote: `print("hi")`

```
print hi
```

ERROR
```
SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?
```

Fix: `print("hi")`



The intern is confused, not angry. Read the **last line**!

AI Corner: meet the assistant

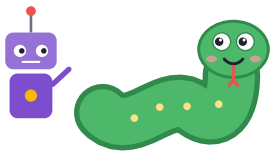
AI CORNER

A good prompt

"I am new to Python. Explain what `print("Hello")` does, in two simple sentences."

Double-check

Run the code yourself. Does it match the explanation?



Recap

✔ Today you learned

- A program is clear steps, built from a few bricks
- LLMs predict the next token. Useful, but check them
- In Colab, `print(...)` shows text and results



Next time: teaching the computer to remember

- Variables: labeled boxes for values
- Numbers, text, and True/False
- Python as a super calculator





3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md